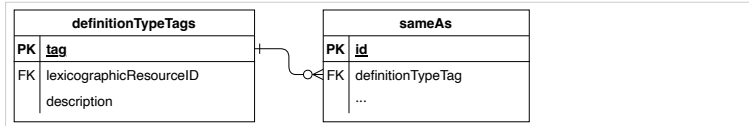


```
<lexicographicResource> dmlex:definitionTypeTag [
  a dmlex:DefinitionTypeTag ;
  dmlex:tag "...";
  dmlex:description "...";
  dmlex:sameAs ... ] .
```

Example 94. Relational database



Example 95. NVH

```
definitionTypeTag: tag
description: ...
sameAs: ...
```

4.2.3 inflectedFormTag

Represents one (of many) possible values for `tag` of `inflectedForm`. Example: [Section A.1.6, "How to use partOfSpeechTag and inflectedFormTag"](#).

Property of

- [lexicographicResource](#)

Properties

- `tag` REQUIRED (exactly one) and UNIQUE. Normalised string. An abbreviation, a code or some other string of text.
- `description` OPTIONAL (zero or one). Normalised string. A human-readable description of what the tag means.
- `for` OPTIONAL (zero or one). Normalised string. A statement about constraints and recommendation on using this tag. This can be a human-readable description such as "for use with German nouns only" or a machine-readable instruction in any format understandable to the DMLex implementation, such as `partOfSpeech=noun AND language=de`.
- `sameAs` OPTIONAL (zero or more).

Example 96. XML

```
<inflectedFormTag tag="..." for="...">
  <description>...</description>
  <sameAs.../>
</inflectedFormTag>
```

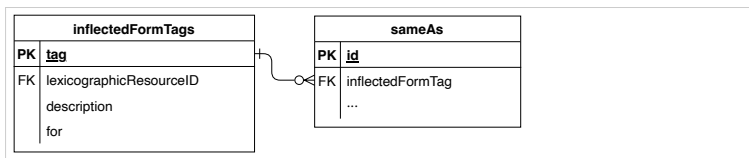
Example 97. JSON

```
{
  "tag": "...",
  "description": "...",
  "for": "...",
  "sameAs": [...]
}
```

Example 98. RDF

```
<lexicographicResource> dmlex:inflectedFormTag [
  a dmlex:InflectedFormTag ;
  dmlex:tag "...";
  dmlex:description "...";
  dmlex:for "...";
  dmlex:sameAs ... ] .
```

Example 99. Relational database



Example 100. NVH

```

formTag: tag
description: ...
sameAs: ...
for: ...
  
```

4.2.4 `labelTag`

Represents one (of many) possible values for `tag` of `label`.

Property of

- [lexicographicResource](#)

Properties

- `tag` REQUIRED (exactly one) and UNIQUE. Normalised string. An abbreviation, a code or some other string of text.
- `description` OPTIONAL (zero or one). Normalised string. A human-readable description of what the tag means.
- `typeTag` OPTIONAL (zero or one). Normalised string. An abbreviation, a code or some other string of text which identifies the type of the label, for example `temporal` for temporal labels (archaic, neologism etc) or `dialect` for labels of dialects. The `labelTypeTag` object type can be used to explain the meaning of the type tags, to constrain which type tags are allowed to occur in the lexicographic resource, and to map them onto external inventories and ontologies.
- `for` OPTIONAL (zero or one). Normalised string. A statement about constraints and recommendation on using this tag. This can be a human-readable description such as “for use with German nouns only” or a machine-readable instruction in any format understandable to the DMLex implementation, such as `partOfSpeech=noun AND language=de`.
- `sameAs` OPTIONAL (zero or more).

Example 101. XML

```

<labelTag tag="..." typeTag="..." for="...">
  <description>...</description>
  <sameAs.../>
</labelTag>
  
```

Example 102. JSON

```

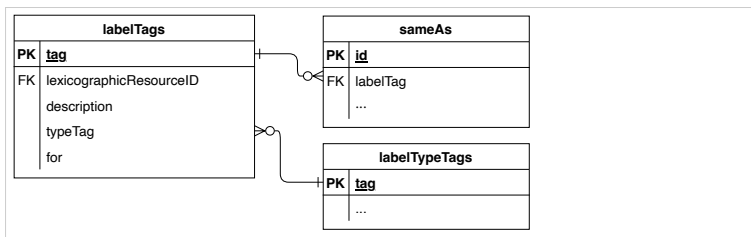
{
  "tag": "...",
  "description": "...",
  "typeTag": "...",
  "for": "...",
  "sameAs": [...]
}
  
```

Example 103. RDF

```

<lexicographicResource> dmlex:labelTag [
  a dmlex:LabelTag ;
  dmlex:tag "...";
  dmlex:description "...";
  dmlex:typeTag "...";
  dmlex:for "...";
  dmlex:sameAs ... ] .
  
```

Example 104. Relational database



Example 105. NVH

```

labelTag: tag
description: ...
sameAs: ...
for: ...
typeTag: ...
  
```

4.2.5 `labelTypeTag`

Represents one (of many) possible values for `typeTag` of `labelTag`.

Property of

- `lexicographicResource`

Properties

- `tag` REQUIRED (exactly one) and UNIQUE. Normalised string. An abbreviation, a code or some other string of text.
- `description` OPTIONAL (zero or one). Normalised string. A human-readable description of what the type tag means.
- `sameAs` OPTIONAL (zero or more).

Example 106. XML

```

<labelTypeTag tag="...">
  <description>...</description>
  <sameAs.../>
</labelTypeTag>
  
```

Example 107. JSON

```

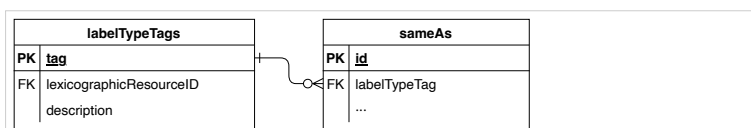
{
  "tag": "...",
  "description": "...",
  "sameAs": [...]
}
  
```

Example 108. RDF

```

<lexicographicResource> dmlex:labelTypeTag [
  a dmlex:LabelTypeTag ;
  dmlex:tag "...";
  dmlex:description "...";
  dmlex:sameAs ... ].
  
```

Example 109. Relational database



Example 110. NVH

```

labelTypeTag: tag
description: ...
sameAs: ...
  
```

4.2.6 `partOfSpeechTag`

Represents one (of many) possible values for `tag` of `partOfSpeech`. Example: [Section A.1.6, "How to use partOfSpeechTag and inflectedFormTag"](#).

Property of

- [lexicographicResource](#)

Properties

- `tag` REQUIRED (exactly one) and UNIQUE. Normalised string. An abbreviation, a code or some other string of text.
- `description` OPTIONAL (zero or one). Normalised string. A human-readable description of what the tag means.
- `for` OPTIONAL (zero or one). Normalised string. A statement about constraints and recommendation on using this tag. This can be a human-readable description such as "for use in German and Czech only" or a machine-readable instruction in any format understandable to the DMLex implementation, such as `language=de OR language=cs`.
- `sameAs` OPTIONAL (zero or more).

Example 111. XML

```
<partOfSpeechTag tag="..." for="...">
  <description>...</description>
  <sameAs.../>
</partOfSpeechTag>
```

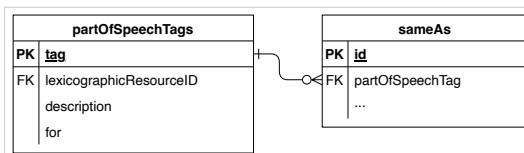
Example 112. JSON

```
{
  "tag": "...",
  "description": "...",
  "for": "...",
  "sameAs": [...]
}
```

Example 113. RDF

```
<lexicographicResource> dmlex:partOfSpeechTag [
  a dmlex:PartOfSpeechTag ;
  dmlex:tag "...";
  dmlex:description "...";
  dmlex:for "...";
  dmlex:sameAs ... ] .
```

Example 114. Relational database



Example 115. NVH

```
postTag: tag
description: ...
sameAs: ...
for: ...
```

4.2.7 `sourceIdentityTag`

Represents one (of many) possible values for `sourceIdentity` of `example`.

Property of

- [lexicographicResource](#)

Properties

- `tag` REQUIRED (exactly one) and UNIQUE. Normalised string. An abbreviation, a code or some other string of text.
- `description` OPTIONAL (zero or one). Normalised string. A human-readable description of what the tag means.

- `sameAs` OPTIONAL (zero or more).

Example 116. XML

```
<sourceIdentityTag tag="..."
  <description>...</description>
  <sameAs.../>
</sourceIdentityTag>
```

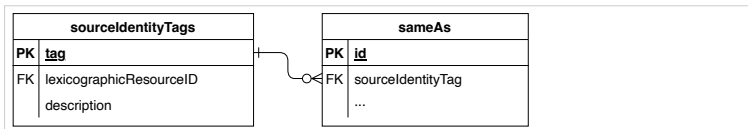
Example 117. JSON

```
{
  "tag": "...",
  "description": "...",
  "sameAs": [...]
}
```

Example 118. RDF

```
<lexicographicResource> dmlex:sourceIdentityTag [
  a dmlex:SourceIdentityTag ;
  dmlex:tag "...";
  dmlex:description "...";
  dmlex:sameAs ... ] .
```

Example 119. Relational database



Example 120. NVH

```
sourceIdentityTag: tag
description: ...
sameAs: ...
```

4.2.8 transcriptionSchemeTag

Represents one (of many) possible values for `scheme` of [transcription](#).

Property of

- [lexicographicResource](#)

Properties

- `tag` REQUIRED (exactly one) and UNIQUE. An IETF language tag.
- `description` OPTIONAL (zero or one). Normalised string. A human-readable description of what the tag means.
- `for` OPTIONAL (zero or one). Normalised string. A statement about constraints and recommendation on using this tag. This can be a human-readable description such as “for use in German and Czech only” or a machine-readable instruction in any format understandable to the DMLex implementation, such as `language=de OR language=cs`.

Comment

- The `transcriptionSchemeTag` does not have a `sameAs` property because the tag itself - which is an IETF language tag - defines fully what the tag means.

Example 121. XML

```
<transcriptionSchemeTag tag="..." for="...">
  <description>...</description>
</transcriptionSchemeTag>
```

Example 122. JSON

```
{
  "tag": "...",
  "description": "...",
}
```

```

    "for": "...
  }

```

Example 123. RDF

```

<lexicographicResource> dmlex:transcriptionSchemeTag [
  a dmlex:PartOfSpeechTag ;
  dmlex:tag "...";
  dmlex:description "...";
  dmlex:for "..."] .

```

Example 124. Relational database

transcriptionSchemeTags	
PK	tag
FK	lexicographicResourceID
	description
	for

Example 125. NVH

```

transcriptionSchemeTag: tag
description: ...
sameAs: ...
for: ...

```

4.2.9 sameAs

Represents the fact that the parent object is equivalent to an item available from an external authority. Example: [Section A.1.7, “Mapping controlled values to external inventories”](#).

Property of

- [definitionTypeTag](#)
- [inflectedFormTag](#)
- [labelTag](#)
- [labelTypeTag](#)
- [partOfSpeechTag](#)
- [sourceIdentityTag](#)

Properties

- `uri` REQUIRED (exactly one) and UNIQUE. The URI of an item in an external inventory.

Example 126. XML

```

<sameAs uri="..."/>

```

Example 127. JSON

Implemented as a string

```

"..."

```

Example 128. Relational database

sameAs	
PK	id
FK	definitionTypeTag
FK	inflectedFormTag
FK	labelTag
FK	partOfSpeechTag
FK	sourceIdentityTag
	uri

Example 129. NVH

```
sameAs: uri
```

4.3 DMLex Linking Module

DMLex's Linking Module can be used to construct *relations* between objects which “break out” of the tree-like parent-and-child hierarchy constructed from datatypes from the Core and from other modules. The Linking Module can be used to create relations between senses which are synonyms or antonyms, between entries whose headwords are homonyms or spelling variants, between senses which represent superordinate and subordinate concepts (eg. hypernyms and hyponyms, holonyms and meronyms), between entries and subentries, between senses and subsenses, and many others.

Each relation is represented in DMLex by an instance of the `relation` datatype. A relation brings two or more *members* together. The fact that an object (such as a sense or an entry) is a member of a relation is represented in DMLex by an instance of the `member` datatype.

The Linking Module can be used to set up relations between objects inside the same lexicographic resource, or between objects residing in different lexicographic resources.

For linking, some type of reference IDs of linked objects are needed (cf. the `ref` property in [Section 4.3.3, “member”](#)). DMLex does not prescribe the exact form of these IDs, however, a recommended method for creating unique IRIs for DMLex objects is available in [Section 3.2.1, “DMLex fragment identification strings”](#), which may be useful especially when linking objects from different lexicographic resources on the Web.

Examples: [Section A.1.13, “Modelling parts and wholes”](#), [Section A.1.14, “Modelling antonyms”](#), [Section A.1.15, “Modelling synonyms”](#), [Section A.1.16, “Modelling variants”](#), [Section A.1.17, “Modelling subsenses”](#), [Section A.1.18, “Modelling subentries \(at subsense level\)”](#), [Section A.1.19, “Modelling subentries \(at sense level\)”](#).

4.3.1 Extensions to `lexicographicResource`

Extends the `lexicographicResource` object type from the [Core](#).

Additional properties

- `relation` OPTIONAL (zero or more)
- `relationType` OPTIONAL (zero or more)

Example 130. XML

```
<lexicographicResource ...>
  ...
  <relation.../>
  <relationType.../>
</lexicographicResource>
```

Example 131. JSON

```
{
  ...,
  "relations": [...],
  "relationTypes": [...]
}
```

Example 132. RDF

```
<#lexicographicResource>
  dmlex:relation ... ;
  dmlex:relationType ... .
```

Example 133. NVH

```
lexicographicResource: ...
  ...
  relation: ...
  relationType: ...
```

4.3.2 `relation`

Represents the fact that a relation exists between two or more objects.

Property of

- `lexicographicResource`

Properties

- `type` REQUIRED (exactly one). Normalised string. Specifies what type of relation it is, for example a relation between synonyms or a relation between a sense and a subsense. Optionally, `relationType` objects can be used to explain those types and to constrain which types of relations are allowed to exist in the lexicographic resource.
- `description` OPTIONAL (zero or one). Normalised string. A human-readable explanation of this relation.
- `member` REQUIRED (two or more).

Example 134. XML

```
<relation type="...">
  <description>...</description>
  <member.../>
</relation>
```

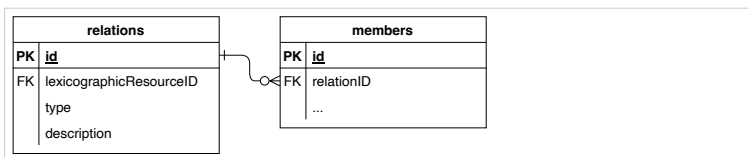
Example 135. JSON

```
{
  "type": "...",
  "description": "...",
  "members": [...]
}
```

Example 136. RDF

```
<#lexicographicResource> dmlex:relation [
  a dmlex:Relation ;
  dmlex:type "...";
  dmlex:description "...";
  dmlex:member ... ] .
```

Example 137. Relational database



Example 138. NVH

```
relation: type
description: ...
member: ...
```

4.3.3 `member`

Represents the fact that an object, such as an entry or a sense, is a member of a relation.

Property of

- `relation`

Properties

- `ref` REQUIRED (exactly one) and UNIQUE (in combination with other unique properties if present). Reference to an object, such as an entry or a sense. The IRI addressing mechanism described in [Section 3.2.1, “DMLex fragment identification strings”](#) can be used (but is not REQUIRED).
- `role` OPTIONAL (zero or one) and UNIQUE (in combination with other unique properties if present). Normalised string. An indication of the role the member has in this relation: whether it is the hypernym or the hyponym (in a hyperonymy/hyponymy relation), or whether it is one of the synonyms (in a synonymy relation), and so on. You can use `memberType` objects to explain those roles and to constrain which relations are allowed to contain which roles, what their object types are allowed to be (eg. entries or senses) and how many members with this role each relation is allowed to have.
- `listingOrder` REQUIRED (exactly one). Number. The position of this member among other members of the same relation. When showing members of the relation to human users (for example: when listing the synonyms in a synonymy relation), the members should be listed in this order. This can be implicit from the serialization.